

Deep Research: Avatrada 57 Topic 004

Engineering Report: 0DTE Regime Detection and Strategy Switching Mechanism

Authored By: Autonomous Technical Researcher **Subject:** Deep-Dive on 0DTE
Gamma Squeeze Detection Logic **Source:** avatrada_57.pdf, Section 4

Executive Summary

This report provides a detailed engineering analysis of the "0DTE Regime Detection" system component. The system's specification is to detect market conditions indicative of a "Gamma Squeeze" by monitoring the ratio of 0-Day-To-Expiry (0DTE) options volume to the total options chain volume for a given underlying. If this ratio exceeds a 40% threshold, the system is designed to switch its active trading strategy from a "Reversion" model to a "Breakout" model.

This analysis deconstructs the core mechanism, provides a detailed implementation strategy including mathematical formulations and code architecture, and performs a critical analysis of potential failure modes, edge cases, and optimizations. The proposed solution is a robust, event-driven system capable of dynamically altering its trading logic in response to real-time market microstructure changes.

1. Technical Deconstruction

The system is designed to identify a specific market regime driven by the intense activity in options expiring on the current trading day. This phenomenon is often associated with "Gamma Squeezes."

1.1. Core Mechanism: The Gamma Squeeze Feedback Loop

A Gamma Squeeze is a positive feedback loop involving options market makers and the underlying asset:

1. **High Call Volume:** A large number of traders buy short-dated, out-of-the-money (OTM) call options. 0DTE options are particularly potent due to their extremely high Gamma.
2. **Market Maker Hedging:** Market makers who sell these calls are short gamma and must remain delta-neutral. To hedge their position, they must buy the underlying asset.
3. **Price Increase:** This hedging-related buying pressure pushes the price of the underlying asset higher.
4. **Gamma/Delta Explosion:** As the asset price rises towards the strike prices of the OTM calls, the Gamma of these options explodes. This causes their Delta to rapidly increase, forcing market makers to buy even *more* of the underlying to maintain their delta-neutral hedge.
5. **The Squeeze:** This cycle of rising prices forcing more buying, which in turn forces prices higher, creates a rapid, accelerating, and often volatile upward price move. This is a strong *trending or momentum-driven* environment.

1.2. The Detection Heuristic

The specified logic does not directly measure net gamma exposure (which is computationally complex and requires proprietary data). Instead, it uses a **volume-based heuristic** as a proxy:

$$\text{Ratio} = \text{Volume_0DTE} / \text{Volume_Total}$$

- **Volume_0DTE** : The sum of all traded contracts (puts and calls) for the option chain that expires on the current date.
- **Volume_Total** : The sum of all traded contracts across all available expiration dates for the underlying.

The trigger condition, `Ratio > 0.40`, posits that when nearly half of the day's entire options activity is concentrated on contracts expiring within hours, the conditions for a gamma-driven feedback loop are highly probable.

1.3. The Strategy Switch

The system's response is a complete change in its operational logic: *

- "Reversion" Strategy (Default):** Assumes that price deviations from a statistical mean (e.g., VWAP, moving average) are temporary and will revert. It profits by selling at highs and buying at lows within a defined range. This strategy fails catastrophically in a strong, directional trend like a squeeze. *
- "Breakout" Strategy (Triggered):** Assumes that a strong trend is in place and will continue. It profits by buying when the price breaks above a key resistance level or shorting when it breaks below support, aiming to ride the momentum.

The 40% threshold is the demarcation line between a market perceived as mean-reverting and one perceived as trending.

2. Implementation Strategy

Building this system requires a real-time data pipeline, precise mathematical calculations, and a robust software architecture to manage the state transition between strategies.

2.1. Data Acquisition

A low-latency, real-time data source for options contract trades is mandatory. *

- Provider:** Services like **Polygon.io**, **Cboe Live**, or **dxFeed** provide WebSocket or API access to tick-level options data. *
- Data Points Needed:** For each options trade on the underlying (e.g., SPY, QQQ), we need: *

- `contract_symbol` (to parse expiration date) *
- `trade_volume` *
- `timestamp` *

- Implementation:** A data handler service would subscribe to the feed for the target underlying. It would aggregate volume in real-time, bucketing it into "0DTE" and "Total" categories.

2.2. Mathematical Formulation

Volume Calculation

Let T be the set of all options trades for the underlying within a given time window (e.g., since market open). For each trade $t \in T$ with volume v_t and expiration date d_t :

```
// Formula for 0DTE Volume
V_0DTE =  $\sum(v_t)$  for all  $t \in T$  where  $d_t == \text{Today}$ 

// Formula for Total Volume
V_Total =  $\sum(v_t)$  for all  $t \in T$ 

// The Regime Detection Ratio
Ratio = V_0DTE / V_Total
```

Altering Mean-Reversion Entry Criteria

The switch is not a parameter tweak; it is a fundamental replacement of the entry logic.

Let P be the current price of the underlying.

1. Regime: REVERSION (Ratio ≤ 0.40)

A typical mean-reversion entry might use Bollinger Bands. * μ_{20} = 20-period Simple Moving Average * σ_{20} = 20-period Standard Deviation * LowerBand = $\mu_{20} - k * \sigma_{20}$ (where k is typically 2)

The entry logic is:

```
// Reversion Entry Logic
IF  $P < \text{LowerBand}$ :
    RETURN "BUY_SIGNAL"
```

2. Regime: BREAKOUT (Ratio > 0.40)

A breakout strategy might use the high of a recent period (e.g., Donchian Channels). * `High_N` = The highest price over the last N periods (e.g., N=20).

The entry logic is:

```
// Breakout Entry Logic
IF P > High_N:
    RETURN "BUY_SIGNAL"
```

The mathematical alteration is the complete substitution of the entry condition from $P < (\mu - k\sigma)$ to $P > \text{High}_N$. The exit logic for each strategy would also need to be switched accordingly (e.g., Reversion exits at the mean, Breakout exits on a trailing stop loss).

2.3. Code Structure: `RegimeSwitch` Class

The Strategy design pattern is ideal here. We define a context (`TradingSystem`), a state manager (`RegimeSwitch`), and separate strategy objects (`ReversionStrategy`, `BreakoutStrategy`).

```
from enum import Enum

class MarketRegime(Enum):
    REVERSION = 1
    BREAKOUT = 2

class RegimeSwitch:
    """
    Monitors options volume to detect the prevailing market regime.
    """
    def __init__(self, threshold: float = 0.40):
        self.threshold = threshold
        self.current_regime = MarketRegime.REVERSION
        print(f"RegimeSwitch initialized. Default regime:
{self.current_regime.name}")

    def update_regime(self, odte_volume: int, total_volume: int) -> bool:
        """
```

```

        Calculates the ODTE volume ratio and updates the regime if necessary.
        Returns True if the regime changed, False otherwise.
        """
        if total_volume == 0:
            return False # Avoid division by zero early in the day

        ratio = odte_volume / total_volume
        new_regime = MarketRegime.BREAKOUT if ratio > self.threshold else
MarketRegime.REVERSION

        if new_regime != self.current_regime:
            print(f"REGIME CHANGE DETECTED: Ratio {ratio:.2f} crossed threshold
{self.threshold}. "
                f"Switching from {self.current_regime.name} to
{new_regime.name}.")
            self.current_regime = new_regime
            return True

        return False

    def get_regime(self) -> MarketRegime:
        return self.current_regime

# --- Example Usage with a Strategy Executor ---

class TradingStrategy:
    """Interface for a trading strategy."""
    def generate_signal(self, price_data):
        raise NotImplementedError

class ReversionStrategy(TradingStrategy):
    def generate_signal(self, price_data):
        # Placeholder for Bollinger Band logic
        print("Executing Reversion logic...")
        return "HOLD" # or "BUY_REVERSION"

class BreakoutStrategy(TradingStrategy):
    def generate_signal(self, price_data):
        # Placeholder for Donchian Channel logic
        print("Executing Breakout logic...")

```

```

        return "BUY_BREAKOUT"

class TradingSystem:
    """
    Main system that holds strategies and uses the RegimeSwitch to decide
    which one to execute.
    """
    def __init__(self):
        self.regime_switch = RegimeSwitch(threshold=0.40)
        self.strategies = {
            MarketRegime.REVERSION: ReversionStrategy(),
            MarketRegime.BREAKOUT: BreakoutStrategy()
        }
        self.active_strategy = self.strategies[self.regime_switch.get_regime()]

    def on_new_data(self, price_data, odte_volume, total_volume):
        """
        Main event loop handler.
        """
        # 1. Update the regime based on the latest volume data
        if self.regime_switch.update_regime(odte_volume, total_volume):
            # If the regime changed, update the active strategy
            self.active_strategy =
self.strategies[self.regime_switch.get_regime()]

        # 2. Generate a signal using the currently active strategy
        signal = self.active_strategy.generate_signal(price_data)
        # 3. (Further logic to execute trades based on the signal)
        print(f"Generated signal: {signal}")

if __name__ == '__main__':
    # --- Simulation ---
    system = TradingSystem()

    # Market starts quiet
    print("\n--- Scenario 1: Normal Market ---")
    system.on_new_data(price_data={}, odte_volume=100_000,
total_volume=400_000) # Ratio = 0.25

```

```
# ODTE volume spikes
print("\n--- Scenario 2: ODTE Volume Spike ---")
system.on_new_data(price_data={}, odte_volume=500_000,
total_volume=1_000_000) # Ratio = 0.50

# Market remains in breakout mode
print("\n--- Scenario 3: Sustained High ODTE Volume ---")
system.on_new_data(price_data={}, odte_volume=600_000,
total_volume=1_200_000) # Ratio = 0.50

# ODTE volume subsides post-lunch
print("\n--- Scenario 4: Market Normalizes ---")
system.on_new_data(price_data={}, odte_volume=550_000,
total_volume=1_500_000) # Ratio = 0.36
```

3. Critical Analysis

While the specified heuristic is a clever and computationally efficient proxy, a production-grade system must account for its limitations.

3.1. Potential Failure Modes & Edge Cases

1. **False Positives (High Volume, No Squeeze):** The 40% threshold can be triggered by events other than a gamma squeeze.
 - **Hedging of Large Positions:** A massive fund rolling a large SPX options position on expiration day can generate enormous ODTE volume without any directional squeeze intent.
 - **High Put and Call Volume:** If volume is high in both puts and calls (e.g., straddle/strangle buying before a catalyst), the net effect could be pinning the price at a high-gamma strike, not a squeeze. The heuristic is blind to the call/put skew.
2. **Regime Flapping (Whipsaw):** If the ratio hovers around 40% (e.g., fluctuating between 39% and 41%), the system could rapidly switch between `REVERSION` and `BREAKOUT` strategies. This "thrashing" would lead to poor trades and high transaction costs.

3. **Data Integrity and Latency:** The system is highly sensitive to the quality of the real-time volume data. A lag in the data feed or a missed burst of volume could cause the system to remain in the wrong regime during a critical market move.
4. **Early/Late Day Skews:** Options volume is not uniformly distributed throughout the day. There are often large bursts at the open and close. The 40% threshold might be breached early in the day on thin total volume, leading to a premature regime switch.

3.2. Optimizations and Enhancements

1. **Introduce Hysteresis (Debouncing):** To prevent regime flapping, implement a two-threshold system.
 - Switch to `BREAKOUT` only if `Ratio > 0.40`.
 - Switch back to `REVERSION` only if `Ratio < 0.35`.
 - This creates a 5% "dead-band" where the regime remains unchanged, preventing rapid oscillations.
2. **Incorporate Call/Put Ratio:** Enhance the heuristic by filtering for directional intent. The trigger could be a composite condition:
 - `Trigger = (Ratio > 0.40) AND (0DTE_Call_Volume / 0DTE_Put_Volume > 1.5)`
 - This ensures the high 0DTE volume is predominantly driven by call buying, which is the primary fuel for a gamma squeeze.
3. **Use a Smoothed Ratio:** Instead of the instantaneous ratio, use a short-term moving average (e.g., 5-minute Exponential Moving Average) of the ratio. This will smooth out noisy, transient volume spikes and provide a more stable signal. `// Logic for Smoothed Ratio current_ratio = odte_volume / total_volume smoothed_ratio = EMA(current_ratio, period=5) IF smoothed_ratio > threshold: ...`
4. **Volume Normalization:** Consider the time of day. The raw ratio could be normalized against the typical ratio for that specific time of day (e.g., 10:00 AM) to avoid false signals caused by predictable intraday volume patterns.
5. **Direct Gamma Measurement (Advanced):** For a more robust system, supplement the volume heuristic with a direct (or estimated) measure of

market-wide Gamma Exposure (GEX). A trigger could require both high 0DTE volume *and* GEX approaching a critical level. This transforms the system from using a proxy to a more direct measurement of the underlying market force.